

AlgebraicPowerSeries documentation

Noé Terrien

June 26, 2026

Contents

1	Introduction	2
1.1	Objectives	2
1.2	Julia version	2
1.3	Dependencies	2
1.4	Examples	2
1.5	Benchmarks	3
2	User guide	3
2.1	PowerSeries	4
2.2	TaylorExpansionSeries	4
2.3	TranslatedSeries	5
2.4	PDESeries and LocalizedPDESeries	6
2.4.1	The "self" series	6
2.4.2	Writing the partial differential equation	6
2.4.3	Constructing PDESeries and LocalizedPDESeries	8
2.4.4	An example	9
3	Detailed documentation	10
3.1	PowerSeries	10
3.1.1	PowerSeries attributes	10
3.1.2	ScalarSeriesSymbol	10
3.1.3	SeriesCoefficient	10
3.1.4	Coefficients indexing : lin, trunc, fullsym	10
3.2	Series defined by Partial Differential Equations	10
3.2.1	SymbolicSeries, EvaluatedSymbolicSeries and SymbolicSeriesEquation	10
3.2.2	NlinearSeriesOperation	10
3.2.3	MultilinearSeriesOperation	10
3.2.4	getindex method on SymbolicSeries	10
3.2.5	getNum and getSymbolics method	10
3.2.6	Evaluating a SymbolicSeries	10
3.2.7	Operations defined on SymbolicSeries	10
3.2.8	Operations defined on EvaluatedSymbolicSeries	10
3.2.9	The compute_coefficients! method	10

1 Introduction

1.1 Objectives

This module introduces an algebraic representation of potentially infinite power series and arrays of power series of any number of variables. These series are represented by `PowerSeries` objects, which contain the instructions to compute the coefficients of the series up to an arbitrarily high order (though that may come at the cost of some algorithmic complexity).

It was primarily developed to compute backstepping kernels (see [1] for details on this method), but it should technically work to solve a number of well-posed Partial Differential Equations. It was tested on multiple examples (see Examples section), correctly reproducing results from `matematica` but possible bugs may remain. Users are encouraged to do their own tests to verify that everything works as expected for their personal use.

1.2 Julia version

This module was developed in Julia 1.12.6

1.3 Dependencies

This module makes use of several pre-existing Julia modules:

- Symbolics: v7.24.2
- TaylorSeries: v0.21.10
- LinearSolve: v3.85.2

These can be installed by typing `]` in the Julia REPL and then using the command `add Symbolics TaylorSeries LinearSolve`

Examples use Pluto notebooks and GLMakie for display

- Pluto: v0.20.27
- PlutoUI: v0.7.83
- GLMakie: v0.13.10

These can be installed by typing `]` in the Julia REPL and then using the command `add Pluto PlutoUI GLMakie`

Tests and benchmarking rely on Julia Test and BenchmarkTools modules

- Test: v1.11.0
- BenchmarkTools: v1.8.0

These can be installed by typing `]` in the Julia REPL and then using the command `add Test BenchmarkTools`

1.4 Examples

Several examples are provided in the form of Pluto notebooks in the "examples" folder. To start Pluto, run `import Pluto` and then `Pluto.run()` in your REPL. Then open the notebook you want. Alternatively, you can also run the "<example>.jl" files directly.

These examples were used to assess the reliability of the module by reproducing the results from `matematica` notebooks presented in [2]

<example>.jl	PDE solved
1-D reaction diffusion equation with space-varying reaction	Example 1 : $\begin{cases} K_{xx}(x, \xi) - K_{\xi\xi} = \frac{\lambda(\xi)+c}{\varepsilon} K(x, \xi) \\ K(x, x) = -\frac{1}{2\varepsilon} \int_0^x (\lambda(\xi) + c) d\xi \\ K(x, 0) = 0 \end{cases}$
coupled 1D kernel	Example 4 : $\begin{cases} \mu(x)K_x^{vv} + \mu(\xi)K_\xi^{vv} = -\mu'(\xi)K^{vv} + c_2(\xi)K^{vu} + [c_4(x) - c_4(\xi)]K^{vu} \\ \mu(x)K_x^{vu} - \varepsilon(\xi)K_\xi^{vu} = -\varepsilon'(\xi)K^{vu} + c_3(\xi)K^{vv} + [c_4(x) - c_1(\xi)]K^{vv} \\ K^{vv}(x, 0) = \frac{q\varepsilon(0)}{\mu(0)} K^{vu}(x, 0) \\ (\varepsilon(x) + \mu(x))K^{vu}(x, x) = -c_3(x) \end{cases}$
2x2 1-D linear hyperbolic system	Inspired from [3] $\begin{cases} \Sigma(x)K_x(x, y) + K_y(x, y)\Sigma(y) = K(x, y)[C(y) - \Sigma'(y)] - C_0(x)K(x, y) \\ K(x, x)\Sigma(x) - \Sigma(x)K(x, x) = C(x) - C_0(x) \\ K(x, 0)\Sigma(0) \begin{pmatrix} q \\ 1 \end{pmatrix} = 0 \end{cases}$ where $\Sigma = \begin{pmatrix} -\varepsilon_1 & 0 \\ 0 & \varepsilon_2 \end{pmatrix}$ with $\varepsilon_1, \varepsilon_2 > 0$ and $C = \begin{pmatrix} c_1 & c_2 \\ c_3 & c_4 \end{pmatrix} \text{ and } C_0 = \begin{pmatrix} c_1 & 0 \\ 0 & c_4 \end{pmatrix}$
localized power series - 1-D reaction diffusion equation	Illustrates the use of LocalizedPDESeries with Example 1
fixing divergence in 1-D reaction diffusion equation with space-varying reaction	Illustrates the use of TranslatedSeries to avoid using LocalizedPDESeries using divergent Example 1. Includes a benchmark to compare performance differences

1.5 Benchmarks

Some benchmarks will be proposed in the future to compare different solvers for PDESeries that are hardware-dependent or trying to use sparsity

2 User guide

This user guide explains how to use the objects defined in "AlgebraicPowerSeries.jl" and "utilities.jl" files to expand functions into their Taylor series, solve Partial Differential Equations (PDE) and build the resulting functions' approximations.

2.1 PowerSeries

`PowerSeries{T,D}` is an abstract type representing power series, or to be more precise, an `Array{D}` of power series. The common interface across every concrete `PowerSeries` allows them to perform a number of operations:

- `compute_coefficients!` – `PowerSeries` is an interface to represent abstract power series. However, most uses require accessing its coefficients. To do so, these coefficients must first be computed up to a given order N by using `compute_coefficients!(ps, N)` where `ps` is a `PowerSeries`.
- `ps.coefficients` – The first way of accessing the previously computed coefficients is to access the `coefficients` attribute of the `PowerSeries`. This array is a `Array{D, Vector{T}}`. Since a `PowerSeries` represents in reality an array of scalar `PowerSeries`, to access the coefficients this way, one must first access the corresponding scalar series. The coefficients are then sorted in increasing order of their coefficients (i.e : $a_1 + a_2x + a_3y + a_4x^2 + a_5xy + a_6y^2 + \dots$). For instance, one could do `ps.coefficients[1,2][3]` to access the coefficient of y in a two variables series x, y at the scalar index $(1, 2)$
- `getindex` – The `getindex` method is another way of accessing a series coefficients. Calling for instance `ps[1,2]` will return a `ScalarSeriesSymbol` that can then be called using the `getindex` method an additional time which will return a `SeriesCoefficient`. Indexing a `ScalarSeriesSymbol` requires using the truncated indices, i.e the series is indexed as $a_{0,0} + a_{1,0}x + a_{1,1}y + a_{2,0}x^2 + a_{2,1}xy + a_{2,2}y^2 + \dots$ (see Coefficients indexing : `lin`, `trunc`, `fullsym` for details on the different ways of indexing). Calling for instance `ps[1,2][1,1]` will return the corresponding `SeriesCoefficient`. Using then `getNum` on this `SeriesCoefficient` will return its value if it has already been computed and throw an error otherwise. Thus, `getNum(ps[1,2][1,1])` is the same as `ps.coefficients[1,2][3]`. See more details on `ScalarSeriesSymbol` and `SeriesCoefficient` in the detailed documentation.
- `build_matrix_elt` – Given a `PowerSeries ps`, it is possible to retrieve the function it represents (or rather its polynomial approximation). To do so, one can use `build_matrix_elt(ps)`. This will return an `Array{D}` of functions of the `PowerSeries`' variables. Additionally, one can eventually provide a second argument N to only compute the polynomial function up to degree N . If N is greater than the order up to which the series has been computed, an error will be thrown
- `build` – This works similarly to `build_matrix_elt` but instead of returning an array of functions, it returns a function that returns arrays. However, this seems to be less performant and the returned functions takes longer to evaluate

Most `PowerSeries` require a type parameter T to be constructed, which is the type the coefficients will be stored as. Some `PowerSeries` will have other uses for it. T must be a concrete type.

In addition, each `PowerSeries` as an ID, stored as the `seriesID::Symbol` attribute. This ID should be different for all series and is used to generate a unique representation of each of the series' coefficients.

2.2 TaylorExpansionSeries

`TaylorExpansionSeries{T,D}` is a subtype of `PowerSeries{T,D}` used to represent a power series that is the Taylor expansion of a function $f : \mathbb{R}^m \rightarrow \mathbb{R}^{n_1 \times n_2 \times \dots \times n_D}$

To create a `TaylorExpansionSeries`, one will need to provide several arguments to its constructor :

- The coefficients' type `T`
- A `seriesID`
- Its `variables`, which must come from the "Symbolics" package. They can be created using `@variables x y z` for instance and must be given as a `VectorNum`
- The function it represents. This must be an `Array{D}` of expressions that use the previously provided variables
- The `center` of the series, as a `Vector` that has the same length as the `variables` Vector.

Here is an example of how to use this constructor to develop the *sin* function around 0:

```
sin_ps = TaylorExpansionSeries{Float64}(:sin, [x], [sin(x)], [0])
```

One can then use it like any other `PowerSeries`, by first computing its coefficients up to a given order N:

```
compute_coefficients!(sin_ps, 30)
```

In the background, this method computes the coefficients using the `TaylorSeries` Julia module, allowing it to very rapidly compute the Taylor expansion of the function.

2.3 TranslatedSeries

Sometimes, one might want to change a `PowerSeries`' center. For instance, this can, in some cases, change the radius of convergence of the series. This is done using a `TranslatedSeries`. Its constructor is pretty simple :

```
TranslatedSeries(seriesID::Symbol, ref::PowerSeries, new_center::Vector)
```

This object stores a referenced `PowerSeries` `ref` and a new center to be able to compute the coefficients of the series around the new center, though at the cost of a loss of precision.

For the `compute_coefficients!` method to work in a mathematically correct way, there must exist a continuous path between both centers, inside the domain of holomorphy of the function represented by the `ref` series. The formula that is used can be deduced from Cauchy's integral formula (see [4]) which gives :

Proposition : If f is holomorphic on some connected open set $\Omega \subset \mathbb{C}^m$, $c, c^* \in \Omega$ and

$$f(z_1, \dots, z_m) = \sum_{\alpha_1=0}^{+\infty} \sum_{\alpha_2=0}^{+\infty} \cdots \sum_{\alpha_m=0}^{+\infty} a_{\alpha_1, \alpha_2, \dots, \alpha_m} (z_1 - c_1)^{\alpha_1} (z_2 - c_2)^{\alpha_2} \cdots (z_m - c_m)^{\alpha_m}$$

on some polydisk centered at $c = (c_1, \dots, c_m)$ and included in Ω . Then there exists a non-empty polydisk $\mathcal{D} \subset \Omega$ centered at $c^* = (c_1^*, \dots, c_m^*)$ such that

$$\forall z \in \mathcal{D}, f(z_1, \dots, z_m) = \sum_{\alpha_1=0}^{+\infty} \sum_{\alpha_2=0}^{+\infty} \cdots \sum_{\alpha_m=0}^{+\infty} a_{\alpha_1, \alpha_2, \dots, \alpha_m}^* (z_1 - c_1^*)^{\alpha_1} (z_2 - c_2^*)^{\alpha_2} \cdots (z_m - c_m^*)^{\alpha_m}$$

And for any of these polydisks, we have $\forall (\alpha_1, \alpha_2, \dots, \alpha_m) \in \mathbb{N}^m$,

$$a_{\alpha_1, \dots, \alpha_m}^* = \sum_{k_1=\alpha_1}^{+\infty} \sum_{k_2=\alpha_2}^{+\infty} \cdots \sum_{k_m=\alpha_m}^{+\infty} \binom{k_1}{\alpha_1} \cdots \binom{k_m}{\alpha_m} a_{k_1, \dots, k_m} (c_1^* - c_1)^{k_1 - \alpha_1} \cdots (c_m^* - c_m)^{k_m - \alpha_m}$$

As mentioned before, this way of computing the coefficients causes a loss of precision, as the coefficients expressions must be truncated. Furthermore, in the above formula, coefficients of a given order are computed using only coefficients of the orders above. Consequently, the loss of precision is more important on coefficients of higher orders. To compute these coefficients up to a given order N , the referenced `PowerSeries` coefficients are first computed up to order N , and then every coefficient's expression is truncated to use only these coefficients. If needed, it is however possible to give an additional named argument `trunc_order` to the `compute_coefficients!` method of `TranslatedSeries` to compute the coefficients of the referenced series up to this `trunc_order` and then use these additional coefficients to compute the `TranslatedSeries`' coefficients up to order N .

2.4 PDESeries and LocalizedPDESeries

`PDESeries` and `LocalizedPDESeries` allow the user to generate series as a result of well-posed partial differential equations.

2.4.1 The "self" series

To construct a series out of a partial differential equation, one has to first write down the partial differential equation, and the unknown for which it has to be solved. This unknown is defined with the help of some particular `ScalarSeriesSymbol` objects. `ScalarSeriesSymbol` usually refer to a `PowerSeries`. However, it is possible to have them refer to a `Symbol` instead. This is used to create `ScalarSeriesSymbol` that refer to a series that has not been created yet. In this case, the `Symbol` used is `:self`, which indicates that the series exists by itself (see `ScalarSeriesSymbol` for more details)

It is possible to create `ScalarSeriesSymbol` or an `ArrayScalarSeriesSymbol` by using the function `selfseries_symbols: selfseries_symbols()` will generate a `ScalarSeriesSymbol` that refers to series `:self` while `selfseries_symbols( $n_1, n_2, \dots, n_D$ )` will generate an `ArrayScalarSeriesSymbol`, D of size $n_1 \times n_2 \times \dots \times n_D$ of that refer to series `:self`

2.4.2 Writing the partial differential equation

Writing the partial differential equations, with its boundary conditions is done using `SymbolicSeries`, for which a number of operations have been defined (multiplication, addition, subtraction, equality, ...) to represent operations on power series. Each `SymbolicSeries` represents a **scalar** power series, making it possible to use Julia native array operations to write multiple equations at once.

`SymbolicSeries`, or `ArraySymbolicSeries` can be constructed in several ways:

- `SymbolicSeries(sss::ScalarSeriesSymbol, center::Vector)` – creates a `SymbolicSeries` from a `ScalarSeriesSymbol`, around a given `center`
- `SymbolicSeries(sss::Array{<:ScalarSeriesSymbol}, center::Vector)` – creates an `ArraySymbolicSeries` around the same `center`
- `SymbolicSeries(ps::PowerSeries)` – creates either a `SymbolicSeries` or an `ArraySymbolicSeries` depending on the size of the `ps` from a given `PowerSeries`.
- `SymbolicSeries{D}(x::Number)` – creates a constant `SymbolicSeries` of D variables. This can be useful when creating an `Array` of `SymbolicSeries` from `SymbolicSeries` and constants at the same time.

To write equations, several operations are implemented between `SymbolicSeries` and/or constants (`SymbolicSeries{D}` represents a `SymbolicSeries` of `D` variables):

operation	first argument	second argument
+	<code>SymbolicSeries{D}</code>	<code>SymbolicSeries{D}</code>
	<code>SymbolicSeries{D}</code>	<code>Number</code>
	<code>SymbolicSeries{D}</code>	<code>SymbolicSeries{0}</code>
-	<code>SymbolicSeries{D}</code>	<code>SymbolicSeries{D}</code>
	<code>SymbolicSeries</code>	<code>Number</code>
	<code>SymbolicSeries</code>	<code>SymbolicSeries{0}</code>
	<code>SymbolicSeries</code>	_____
*	<code>SymbolicSeries</code>	<code>Number</code>
	<code>SymbolicSeries{1}</code>	<code>SymbolicSeries{1}</code>
/	<code>SymbolicSeries</code>	<code>Number</code>

All operations `+`, `-`, `*` involving a `SymbolicSeries` and a constant (either `Number` or `SymbolicSeries{0}`) can be used the other way around.

This set of operations is however quite limited. To access a wider range of operations, one needs to be able to evaluate `SymbolicSeries` in some variables and / or constants. If `s` is a `SymbolicSeriesD`, this can simply be done using `s(x1, x2, ..., xD)` where `x1, x2, ..., xD` can either be variables (obtained with the `@variables` macro) or constants.

Once this is done, one obtains an `EvaluatedSymbolicSeries` and a number of additionnal operations:

operation	first argument	second argument
+	<code>EvaluatedSymbolicSeries{D}</code>	<code>EvaluatedSymbolicSeries{D}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>SymbolicSeries{D}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>Number</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>SymbolicSeries{0}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>EvaluatedSymbolicSeries{0}</code>
-	<code>EvaluatedSymbolicSeries{D}</code>	<code>EvaluatedSymbolicSeries{D}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>SymbolicSeries{D}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>Number</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>SymbolicSeries{0}</code>
	<code>EvaluatedSymbolicSeries{D}</code>	<code>EvaluatedSymbolicSeries{0}</code>
*	<code>EvaluatedSymbolicSeries</code>	_____
	<code>EvaluatedSymbolicSeries</code>	<code>Number</code>
/	<code>EvaluatedSymbolicSeries</code>	<code>EvaluatedSymbolicSeries</code>
	<code>EvaluatedSymbolicSeries</code>	<code>Number</code>
∂_x where ∂_x is a <code>Differential</code> object from <code>Symbolics</code>	<code>EvaluatedSymbolicSeries</code> where the variable of the <code>Differential</code> is among the series variables <code>Array<:EvaluatedSymbolicSeries</code>	_____ _____

$\int$	<code>EvaluatedSymbolicSeries</code> <code>EvaluatedSymbolicSeries</code> <code>EvaluatedSymbolicSeries{1}</code> (will also work with arrays of EvaluatedSymbolicSeries)	x , a variable of the series a, b, x where a is the lower bound, b the upper bound (both constants) and x a variable of the series a, b , where a and b must have at most one variable
$\sim$	<code>EvaluatedSymbolicSeries</code> <code>EvaluatedSymbolicSeries</code>	<code>EvaluatedSymbolicSeries</code> <code>Number</code>

Once again, if it makes sense mathematically, arguments can be used in the other order.

The operator $\sim$ is used to create `SymbolicSeriesEquation`, i.e equations between series. That can then be used to construct `PDESeries` and `LocalizedPDESeries`

2.4.3 Constructing PDESeries and LocalizedPDESeries

The constructors for the `PDESeries` and `LocalizedPDESeries` are exactly the same. They have the following arguments:

- `{T}` – The type parameter of the coefficients' type
- `seriesID::Symbol` – The series' ID
- `variables::VectorNum` – The variables of the series
- `center::Vector` – The center of the series
- `unknown::ScalarSeriesSymbol` or `unknown::Array{<:ScalarSeriesSymbol}` – The `ScalarSeriesSymbol` referring to `:self` series that were used in the equations
- `equations::Vector{<:SymbolicSeriesEquation}` – The partial differential equation(s) and its boundary conditions

Additionally, if the unknown series in your equations is integrated, you will need to provide an additional argument: `maxIntegrationOrders::Vector{Int}`. For each equation, indicate how many times the unknown series has been integrated at most. This will be used to deduce which equations should be solved when computing the coefficients.

The main difference between `PDESeries` and `LocalizedPDESeries` comes from the algorithmic complexity of the coefficients' computation, and when they can be used: Some operations, such as evaluating a `SymbolicSeries` at the wrong points or variables, or integrating between "non-natural" bounds will make the resulting series' coefficients depend on an infinite number of coefficients. This implies that to be computed numerically, the coefficients expressions will have to be truncated up to a certain order. In this case, `LocalizedPDESeries` should be used.

Otherwise, `PDESeries` can be used. When using `PDESeries`, coefficients can be computed with their exact value order by order, this results in the algorithmic complexity being $O(N \times N^D)$ where N is the order up to which the coefficients are computed and D the number of variables.

When computing the coefficients of a `LocalizedPDESeries`, only approximations of the coefficients can be obtained and they all have to be computed at the same time, thus resulting in an algorithmic complexity of $O(N^{2 \times D})$. Thus, whenever possible, one should use `PDESeries` rather than `LocalizedPDESeries`.

Both `compute_coefficients!` method offer two named arguments: `verbose`, an integer between 0 and 3 that is 0 by default and `solver` that corresponds to `LinearSolve`' default solver by default but can be replaced by any other solver from this module. Note that for these two `PowerSeries`, the type parameter `T` is used to convert the coefficients of the `LinearProblem` that has to be solved to the type `T`. Thus if one wants to use a GPU solver for instance, it is possible to choose `T=Float32` to make the computation faster (at the cost of precision).

Finally, note that computing the coefficients of the `PDESeries` will make the `SeriesCoefficient` stored in the unknown `ScalarSeriesSymbol` change so that they refer to the `PDESeries` instead of `:self`. Thus, avoid using the same unknown `ScalarSeriesSymbol` multiple times if you don't want that. (See `ScalarSeriesSymbol`, `SeriesCoefficient` and The `compute_coefficients!` method for more details)

2.4.4 An example

Here is a full example using `PDESeries`:

```

1
2 # variables and differential operators
3 @variables x y
4 Dx, Dy = Differential(x), Differential(y)
5
6 # parameters
7 q = 1
8 N = 30
9
10 E_ps = TaylorExpansionSeries{Float64}(:E, [x], [-1.2-x^3;0;;0;1.5+x^2], [0])
11 compute_coefficients!(E_ps, N+1);
12 E = SymbolicSeries(E_ps)
13
14 C_ps = TaylorExpansionSeries{Float64}(:C, [x],
15   ↪ [3*cos(x);1+2*exp(x);sin(2*x);1/(3+x^2)], [0])
16 compute_coefficients!(C_ps, N)
17 C = SymbolicSeries(C_ps)
18 C0 = SymbolicSeries{1}[C[1,1];0;;0;C[2,2]]
19
20 # unknowns
21 unknowns = selfseries_symbols(2,2)
22 K = SymbolicSeries(unknowns, [0,0])
23
24 # PDEs and boundary conditions
25 PDEs = E(x)*Dx(K(x,y)) + Dy(K(x,y))*E(y) .~ K(x,y)*(C(y)-Dy(E(y))) - C0(x)*K(x,y)
26 BC1 = K(x,x)*E(x) - E(x)*K(x,x) .~ C(x) - C0(x)
27 BC2 = K(x,0)*E(0)*[q, 1] .~ 0
28
29 # PDESeries and computing the coefficients
30 K_ps = PDESeries{Float64}(:K, [x,y], [0,0], unknowns, [BC1...; BC2...; PDEs...;])
31 compute_coefficients!(K_ps, N)

```

3 Detailed documentation

3.1 PowerSeries

3.1.1 PowerSeries attributes

3.1.2 ScalarSeriesSymbol

3.1.3 SeriesCoefficient

3.1.4 Coefficients indexing : lin, trunc, fullsym

3.2 Series defined by Partial Differential Equations

3.2.1 SymbolicSeries, EvaluatedSymbolicSeries and SymbolicSeriesEquation

3.2.2 NlinearSeriesOperation

3.2.3 MultilinearSeriesOperation

3.2.4 getindex method on SymbolicSeries

3.2.5 getNum and getSymbolics method

3.2.6 Evaluating a SymbolicSeries

3.2.7 Operations defined on SymbolicSeries

3.2.8 Operations defined on EvaluatedSymbolicSeries

3.2.9 The compute_coefficients! method

References

- [1] M. Krstic and A. Smyshlyaev, *Boundary Control of PDEs: A Course on Backstepping Designs*, SIAM, Philadelphia, 2008.
- [2] R. Vazquez, G. Chen, J. Qiao, and M. Krstic, “The power series method to compute backstepping kernel gains: Theory and practice,” in *Proc. 63rd IEEE Conference on Decision and Control (CDC)*, 2024.
- [3] J.-M. Coron, R. Vazquez, M. Krstic, and G. Bastin, “Local exponential H^2 stabilization of a 2×2 quasilinear hyperbolic system using backstepping,” *SIAM J. Control Optim.*, vol. 51, pp. 2005–2035, 2013.
- [4] Jiří Lebl, “Tasty Bits of Several Complex Variables, a whirlwind tour of the subject”